Neuro-symbolic approaches to the SAT problem

Pattern Recognition &
Reinforcement Learning
(survey with coding project)

Donato Meoli

SAT Problem

- **SAT**: is there an assignment satisfying a Boolean formula (CNF)? First problem proven NP-complete; core of verification, planning, synthesis.
- Modern solvers are **CDCL**: decide a variable, unit-propagate, on a conflict learn a clause and backtrack.
- The **branching heuristic** (e.g. VSIDS) picks the decision variable — a natural target for machine learning.
- *This project*: survey + implementation of two neuro-symbolic branching approaches, modernised to a single PyTorch stack and reproduced.

- NeuroSAT
- Background
  - Message Passing Neural Networks (MPNNs)
- Model
- Training
- Results

(ISPR – survey)

Selsam et al., *Learning a SAT Solver from Single-Bit Supervision*.

Gilmer et al., *Neural Message Passing for Quantum Chemistry*.

NeuroSAT

NeuroSAT

- **Goal**: can a neural network learn to solve SAT?
- An MPNN trained *only* as a SAT/UNSAT classifier (one-bit supervision) nonetheless learns to find satisfying assignments.
- At test time, for a satisfiable instance the network guesses UNSAT with low confidence until it "finds" a solution, then converges to SAT.
- Survey item only — not implemented here, but the graph view it introduces is the basis of Graph-Q-SAT.

- AlphaZeroSAT
- Background
  - Monte Carlo Tree Search (MCTS)
- Model
- Training
- Results

(ISPR + RL – survey with coding)

Wang & Rompf, *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero.*

AlphaZeroSAT

AlphaZeroSAT — Model

- **Goal**: can Alpha(Go)Zero learn a branching heuristic for SAT?
- State: CNF as a fixed-size **clauses** $\times$ **variables** matrix, entries encoding literal polarity ($(1,0)$ / $(0,1)$).
- A **CNN** with a policy head π (over $2\,n_{\text{var}}$ actions) and a value head $v \in [-1,1]$; invalid actions are masked.
- MiniSat is extended into a *game* env supporting MCTS **simulations** (hypothetical roll-outs without mutating the state).

AlphaZeroSAT — Training

- **Self-play**: MCTS, guided by (π, v), produces improved visit distributions; tuples (state, MCTS-policy, outcome) are stored.
- **Supervised** update with the AlphaZero loss

$$\mathcal{L} = -\sum_a \pi_a^{\mathsf{MCTS}} \log \pi_a + (z - v)^2 + c\|\theta\|_2^2.$$

- Dataset: uniform-random 3-SAT, 20 vars / 91 clauses (`uf20-91`), 32 train / 200 test.

AlphaZeroSAT — This work (modernisation)

- Ported TensorFlow 1.x → **PyTorch**: networks (`models_torch`), trainer (`alphazero_torch`), self-play driver (`main_torch`).
- **Removed the GSL dependency** from the C++ env: Dirichlet noise reimplemented with C++11 `<random>` ⇒ builds with only `g++`/`zlib`, no `sudo`.
- Cleaned the fork (dropped unused OpenAI-baselines code), **runs end-to-end** locally on the modern stack.
- *Limitation*: the fixed-size CNN does not generalise across problem sizes — motivating the graph approach.

- GraphQSAT
- Background
  - Deep Q-Learning (DQN)
  - Graph Neural Networks (GNNs)
- Model
- Graph Attention Networks (GATs)
- Training
- Results

(ISPR + RL – survey with extra coding)

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*

Battaglia et al., *Relational inductive biases, deep learning, and graph networks.*

GraphQSAT

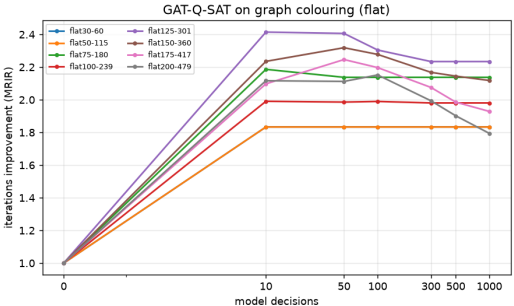
Graph-Q-SAT — Model

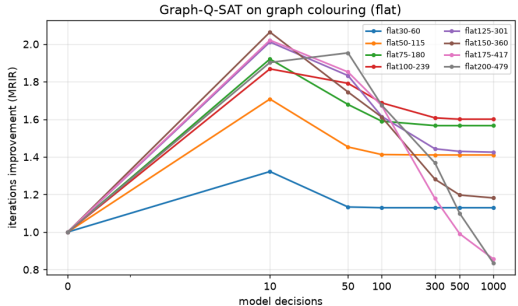
- **Goal**: can DQN with a GNN learn a *generalizable* branching heuristic?
- State: **bipartite graph** (variable / clause nodes); node features 2-D one-hot, edge features = literal polarity, empty global.
- Q-function: **encode–process–decode** graph network; action = $\arg\max Q$ over variable nodes.
- Reward $-p$ per step, 0 at termination; metric = **MRIR** (median MiniSat-iters / model-iters).

GAT-Q-SAT — attention variant (extra coding)

- Inject **Graph Attention** (`GATConv`, edge-aware, multi-head) into the core message-passing block.
- Attention weights neighbouring clauses/variables — useful when the instance has an exploitable sub-structure.
- Same DQN training; toggled with `-use_attention`.
- *Note*: the field later moved to attention/Graph-Transformers (NeuroBack, ICLR'24) — this variant anticipated that direction.

Results — graph colouring (MRIR vs model decisions)



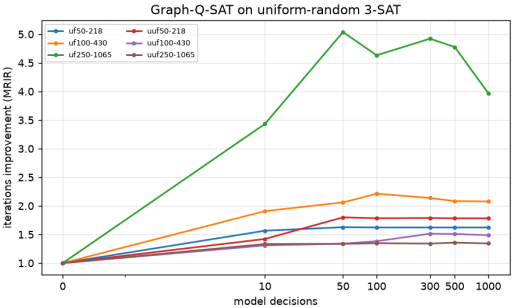


GAT-Q-SAT: strong & stable, also on large instances

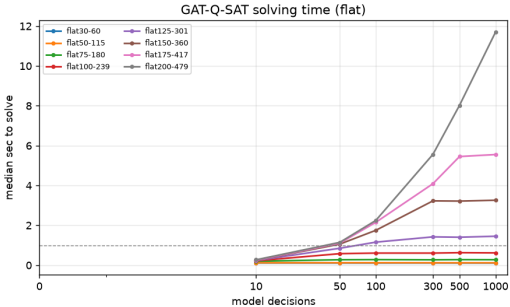
Graph-Q-SAT: degrades on large instances at high caps

Generated locally from `runs/*/*.tsv` (`make_plots.py`) — no more Google Sheets.

Results — random 3-SAT and solving time



On random 3-SAT, plain Graph-Q-SAT is strongest ($\sim 5\times$)



Wall-clock time vs. model decisions

Finding: attention helps where there is structure to infer (graph colouring); on near-uniform random SAT it does not.

Results — exact reproduction (this work)

- Re-ran the *original trained checkpoints* on the modern stack (latest torch / PyTorch Geometric / numpy $\geq$2 / gymnasium).
- MRIR matches the committed logs **exactly** — e.g. `flat30-60`, cap 500:

Model	MRIR (median)	committed
Graph-Q-SAT	1.833	1.833
GAT-Q-SAT	1.667	1.667

- Drop-in changes only (scatter, env construction, GATConv key remap) — **semantics preserved**.

Conclusions

- Two neuro-symbolic SAT methods modernised to one self-contained **PyTorch** stack; brittle deps (TF1.x, GSL, torch-scatter/sparse, pinned gym) removed.
- Results **reproduced exactly**; figures regenerated locally.
- **Graph attention** helps on structured problems (graph colouring), not on uniform-random SAT.
- The fixed-size CNN (AlphaZeroSAT) cannot generalise across sizes — the gap graph methods and later work (NeuroCore/Comb/Back) address.

Code, report and reproducible plots in the project repository.